

Easy system call tracing for Plan 9

Ron Minnich
rminnich@sandia.gov

ABSTRACT

Tracing system calls makes debugging easy and fast. On Plan 9, traditionally, system call tracing has been implemented with *acid*(1). New systems do not always implement all the capabilities needed for Acid, particularly the ability to rewrite the process code space to insert breakpoints. Architecture support libraries are not always available for Acid, or may not work even on a supported architecture. The requirement that Acid's libraries be available can be a problem on systems with a very small memory footprint, such as High Performance Computing systems where every Kbyte counts. Finally, Acid tracing is inconvenient in the presence of forks, which means tracing shell pipelines is particularly troublesome. The strace program available on most Unix systems is far more convenient to use and more capable than Acid for system call tracing. A similar system on Plan 9 can simplify troubleshooting. We have built a system calling tracing capability into the Plan 9 kernel. It has proven to be more convenient than strace in programming effort. One can write a shell script to implement tracing, and the C code to implement an strace equivalent is several orders of magnitude smaller.

Introduction

System call tracing is an important and effective way to debug programs. With a system call tracer, one can find writes to bad file descriptors (as in openssh on Plan 9); trace the auth transactions and see all the error cases; and even find an error in memory initialization. It is particularly convenient to follow forks, such as shell pipelines, and see errors in the interactions between programs in the pipeline. Programs that fail silently and mysteriously can often be understood once traced with a system call tracer. Basic programming errors can be found and resolved.

Probably the most widely used tracer is *strace*, which uses the *ptrace* system call. Ptrace is quite low level, providing little more than a start/stop and peek/poke interface. One word of data is returned at a time. Thus, once a process stops, programs using ptrace must take on the task of examining a great deal of machine state to figure out where the program is executing. For a simple command such as `/bin/date`, which performs 146 system calls on Linux, strace must perform over 2400 ptrace system calls.

Sandia is a multiprogram laboratory operated by Sandia Corporation, a Lockheed Martin Company, for the United States Department of Energy's National Nuclear Security Administration under contract DE-AC04-94AL85000. SAND- 2011-6029C



Sandia National Laboratories

This simplicity is the cause of another problem: `ptrace` must interpret the binary, and it can get very complicated. The need for interpretation may also explain why `strace` is so large. `Sloccount` reports it as follows:

Total Physical Source Lines of Code (SLOC)	= 58,044
Development Effort Estimate, Person-Years (Person-Months)	= 14.22 (170.67)
(Basic COCOMO model, Person-Months = $2.4 * (KSLOC^{**1.05})$)	
Schedule Estimate, Years (Months)	= 1.47 (17.63)
(Basic COCOMO model, Months = $2.5 * (person-months^{**0.38})$)	
Estimated Average Number of Developers (Effort/Schedule)	= 9.68
Total Estimated Cost to Develop	= \$ 1,921,263

An expensive program! As frequently happens, the problem with the Unix world is that the interface is wrong. The 40-year-old `ptrace` system call requires a complicated invocation and gives back almost no data. To create useful data, software must do a great deal of interpretation, matching data in the process address space with the return from the `ptrace` system call. It is a very complex process.

The contrast for the *ratrace* tool, which uses the new Plan 9 system call tracing support, is revealing. *Ratrace* uses two system calls for each system call in the traced program, not 15. *Ratrace* is also much smaller:

Total Physical Source Lines of Code (SLOC)	= 151
Development Effort Estimate, Person-Years (Person-Months)	= 0.03 (0.33)
(Basic COCOMO model, Person-Months = $2.4 * (KSLOC^{**1.05})$)	
Schedule Estimate, Years (Months)	= 0.14 (1.64)
(Basic COCOMO model, Months = $2.5 * (person-months^{**0.38})$)	
Estimated Average Number of Developers (Effort/Schedule)	= 0.20
Total Estimated Cost to Develop	= \$ 3,712

The significance of this result is not so much in its size as what it shows about the importance of getting the kernel interface right. The kernel has knowledge of process state that, in the Unix case, has to be carefully reconstructed from the limited information provided by `ptrace`; a great deal of information is lost as it is passed from the kernel to the user program. In the case of *ratrace*, the kernel provides much richer information that can be used directly and easily, in many different languages, including shell scripts.

In the rest of this paper, we will describe the usage and implementation of the tracing software.

Usage

The most common way to use the system call tracing is via the `ratrace(1)` program. We now show several scenarios in which *ratrace* can be used to understand and debug programs.

`trace /bin/date`

Ratrace can be invoked to trace a process or an individual command. To trace a `pid`, the user types:

```
ratrace pid
```

By far the most common usage is to trace a command, e.g.:

```
ratrace -c /bin/date
```

Shown below is a trace of /bin/date.

```
term% ratrace -c /bin/date
79 date Open 0x19ae 0000702c/"dev/bintime" 00000020 = 3 "" 0x11ce7c2536c76f20 0x11ce7c253736a7c8
79 date Pread 0x19e4 3 0fffff00/"..|<P|." 8 0 = 8 "" 0x11ce7c253c4daa18 0x11ce7c253c5080a8
79 date Open 0x1f58 000077c9/"env/timezone" 00000000 = 4 "" 0x11ce7c2540848d68 0x11ce7c2540884a70
79 date Pread 0x1940 4 0ffff874/"EST.-18000.EDT.-14400....9943200...25664400...41392800...5771880" 1680
    -0x1 = 1518 "" 0x11ce7c2544633e98 0x11ce7c2544661cf8
79 date Close 0x1fc8 4 = 0 "" 0x11ce7c2546831540 0x11ce7c254684e230
79 date Pwrite 0x47d0 1 0ffffde8/"Sun.Aug.29.12:30:43.EDT.2010." 29 -0x1Sun Aug 29 12:30:43 EDT 2010
    = 29 "" 0x11ce7c2549dff7f8 0x11ce7c254d3535a8
79 date Open 0x1a81 0000759c/"#c/pid" 00000000 = 4 "" 0x11ce7c254fccbcf0 0x11ce7c254fcf0eb0
79 date Pread 0x1940 4 0fffff08/".....79." 20 -0x1 = 12 "" 0x11ce7c2553c6e178 0x11ce7c2553c8f4b8
79 date Close 0x1aaf 4 = 0 "" 0x11ce7c2557dbc1c0 0x11ce7c2557dd09e0
79 date Exits 0x1717 0/"cwrite: /proc/79/syscall: failed 12 bytes: process exited
term%
```

For each system call, ratrace prints out the pid, binary name, system call name, program counter, system call args, result, errstr, and entry and exit time. The only part that varies between different calls are the number of arguments. Data arguments are printed in an address/data format. The format is determined by the kernel, not the ratrace program.

Going through the execution call by call, one can see that the first system call is an open of /dev/bintime, mode 0x20, which returns fd 3. The errstr is empty (the system call succeeded), and the time was 0x11ce7c253736a7c8 - 0x11ce7c2536c76f20 or 7289088 nanoseconds.

The program next reads the timezone file, processes the bintime and timezone internally, and writes the output via Pwrite. Finally, the program does a getpid() by opening, reading, and closing the pid file; and ends by calling Exit.

A point of interest is that /bin/date on Plan 9 is 10 system calls; on Linux, it is 146. Three of these calls could be removed by eliminating the read of the pid file, since, as we will see below, a process pid is available on the stack.

In the case of date, someone coming in from the Unix world, trying to understand how Plan 9 programs find the time, could learn without much effort how it is done. System call tracing can speed the process of learning how to use an unfamiliar operating system.

Identifying a vx32 problem on Linux

Following a Plan 9 source code update, it was found that the date command was no longer working correctly. Instead of printing the date, it would just sit. At some point, in disgust, users would hit return and be awarded with the date. The behavior made no sense.

Ratrace facilitated a quick diagnosis and resolution of the problem. The trace above is on OSX, and uses a version of 9vx built from <http://bitbucket.org/rminnich/vx32>. For the same 9vx kernel code on Linux, with the same root file system, the trace was very different:

```
75 date Pread 0x19f6 0 0ffffee0/" 8 0 = 1 "" 0x11ce7e00be3eab80 0x11ce7e0113680020
75 date Close 0x1a30 0 = 0 "" 0x11ce7e01142c0ba0 0x11ce7e01142c80d0
75 date Open 0x1a89 0000702c/" /dev/bintime" 00000020 = 0 "" 0x11ce7e0114ccb0a0 0x11ce7e0114fd78e8
75 date Pread 0x19f6 0 0ffffee0/"..~....." 8 0 = 8 "" 0x11ce7e0115821238 0x11ce7e011582d970
75 date Open 0x1ffb 000077c9/" /env/timezone" 00000000 = 3 "" 0x11ce7e01162aa290 0x11ce7e01162bb018
75 date Pread 0x1940 3 0ffff854/"EST.-18000.EDT.-14400....9943200...25664400...41392800...5771880"1680
    -0x1 =1518 "" 0x11ce7e01169bde8 0x11ce7e01169c6f60
75 date Close 0x206b 3 = 0 "" 0x11ce7e011730b468 0x11ce7e0117310288
75 date Pwrite 0x4873 1 0ffffdc8/"Sun.Aug.29.13:04:46.EDT.2010." 29 -0x1Sun Aug 29 13:04:46 EDT 2010
    = 29 "" 0x11ce7e0117aa0700 0x11ce7e0117f8b3a0
75 date Open 0x1b24 0000759c/"#c/pid" 00000000 = 3 "" 0x11ce7e01189a4ad0 0x11ce7e01189b8350
75 date Pread 0x1940 3 0ffffee8/".....75." 20 -0x1 = 12 "" 0x11ce7e0118fef070 0x11ce7e0118ffc360
75 date Close 0x1b52 3 = 0 "" 0x11ce7e0119660af8 0x11ce7e0119666ca0
75 date Exits 0x1717 0/"cwrite: /proc/75/syscall: failed 12 bytes: process exited
term%
```

Tracing turns a mysterious hang into a less mysterious problem: the program is reading 8 bytes from stdin, hanging until the read is done. It seems at some point the user became impatient and hit return. That can be seen in the return of 1 from the Pread. Since one byte is not a valid return from /dev/bintime, /bin/date closes fd 0 (almost always a bad idea!) and then opens /dev/bintime, and in that case reads 8 bytes back. The read from fd 0 is clearly wrong; the cause is not at all obvious.

Further examination showed that any program which used nsec on the Linux-based 9vx would hang on a read from fd 0. Date would hang, cpu would hang, factotum would hang: all users of nsec() would hang, until one hit an extra return. Additionally, acme would not exit correctly. There was a lot wrong with code built from source.

The initial trace allowed very quick identification of a focus area, since many programs could be traced to find ones that worked and ones that failed. As it happened, the issue was that the 64-bit build of vx32 results in an incorrectly sized Tos struct. In essence, with a 64-bit build of vx32, one is running a heterogeneous system, with 32-bit binaries hosted by a 64-bit kernel. In this case, the value of Tos->pid was in the wrong place on the stack for the 32-bit binaries and all programs using Tos->pid read the pid as 0. This mistake led to lots of other problems, as might be imagined were getpid() to start returning 0.

All structs that are shared between kernel and user space must be carefully defined. In this case, the kernel is on a 64-bit system and built by gcc, while the user programs are 32-bits and built with 8c. There are four ways for this combination to run into trouble. While there are very few shared structs in Plan 9, the Tos struct is a crucial one and it needed to be redefined in the vx32 tree so its size and layout matched that of the 32-bit user mode binaries.

openssh hangs

Users of openssh reported that in many cases, it would hang, with no initial output. A trace of ssh is shown below.

```
82 ssh Open 0xf9473 00123dc4/"/dev/cons" 00000000 = 5 "" 0x11ce7dcb6fb1fa10 0x11ce7dcb70a6ae98
82 ssh Pwrite 0xfb50f 2 001a7704/"The.authenticity.of.host.'dancer.ca.sandia.gov.(146.246.246.1)'." 86 -0x1
    = 86 "" 0x11ce7dcb739e5f10 0x11ce7dcb793b8d08
82 ssh Pwrite 0xfb50f 2 001a7704/"RSA.key.fingerprint.is.43:ef:41:6f:6f:bd:28:5c:58:a5:6f:09:d6:67" 72 -0x1
    = 72 "" 0x11ce7dcb7b995ee0 0x11ce7dcb7f189e78
82 ssh Pwrite 0xfb50f 2 001a7704/"Are.you.sure.you.want.to.continue.connecting.(yes/no)?." 55 -0x1
    = 55 "" 0x11ce7dcb818c0b90 0x11ce7dcb86f56400
82 ssh Pread 0xfb793 5
```

This is the point at which the program needs to ask about continuing. Note the output to fd 2 on the 2nd through 4th lines via the Pwrite system call. In an older version, due to a bug which seems to have been fixed, the output was to a bad fd. The problem was not due to a hang; it was that a message was printed to an invalid fd, and then the program waited for a response which the user did not know need to be made. The program was waiting for the user to type "yes" or "no".

This bug persisted for several years. The ssh code is complex and impenetrable; people got in the habit of taking a look and giving up. System call tracing allowed us to resolve the problem almost instantly, without having to attempt to learn the ssh code.

troff failure

While preparing this paper we were having trouble with troff. The troff command resulted in the rather useless error:

```
postnote 144: sys: write on closed pipe pc=0x0001f89c
```

To resolve the problem we traced the rc from which we ran the command. Tracing rc, instead of a specific command, can sometimes be the most effective approach, especially where a pipeline is involved. The trace showed that the real problem was that /bin/gs was missing, and the error was from page .

Implementation

ratrace consists of a very simple patch to the kernel and a user program. Ratrace is currently only available in the 9k and 9vx kernels, though a patch has been submitted for the standard kernel, and has been tested on the ARM. You can view the changes to vx32 at <https://bitbucket.org/rminnich/vx32> and the 9k changes at <https://bitbucket.org/ericvh/hare>. We for the most part describe the vx32 changes here; the 9k changes are arguably better but we expect more users to see the vx32 kernel, not the 9k kernel.

Kernel changes

Processes need to be marked when they are being traced. Once marked, they halt in the kernel at each system call and wait for the command to continue. The commands are issued to the ctl file for the proc.

To indicate tracing we add a new enum, `Proc_tracesyscall`, to the enums in `portdat.h`.

We also add support for syscall tracing to `devproc.c`. If the command `startsyscall` is written to a process ctl file, then the `procctl` struct member is set to `Proc_tracesyscall`.

The only other pieces needed are a way to control the process tracing and a way to get the data from the kernel. We address the data first, since it is simple.

The tracing information is kept as a string in the proc structure. It is contained in a Proc struct member, `char*syscalltrace` and that information may be read at `/proc/<pid>/syscall`. The code to support reading it is found in the `procread` function in `a/devproc.c` in 9vx:

```
case Qsyscall:
    if(!p->syscalltrace)
        return 0;
    n = readstr(offset, a, n, p->syscalltrace);
    return n;
```

The actual tracing support is in `trap.c`. In the `syscall` function, there are functions to fill in the `syscalltrace` string at `syscall` entry and exit. The entry code is:

```
if(up->procctl == Proc_tracesyscall){
    up->procctl = Proc_stopme;
    syscallprint(ureg);
    procctl(up);
    if(up->syscalltrace)
        free(up->syscalltrace);
    up->syscalltrace = nil;
    startnsec = todget(nil);
}
```

Basically, the code tests if the process is traced and, if so, sets `procctl` to `Proc_stopme`, which will halt the process; calls `syscallprint`, to set `up->syscalltrace` string so the tracing program can read it; and then stop. This stop allows the tracing system to gain control. The process will block until the tracing program sends a command to start again. Note that the process might not be restarted with tracing enabled; the tracing program can examine its actions and decide to set the traced process free.

Note that the tracing program is not required to read the `syscalltrace` string; no matter what it does, the string is freed when the process restarts. The `syscall` exit code is very similar:

```
if(up->procctl == Proc_tracesyscall){
    stopnsec = todget(nil);
    up->procctl = Proc_stopme;
    retprint(ureg, scallnr, startnsec, stopnsec);
    s = splhi();
    procctl(up);
    splx(s);
    if(up->syscalltrace)
        free(up->syscalltrace);
    up->syscalltrace = nil;
}
```

A fraction of the print functions are shown below. `Syscallprint`, in the `vx32` version, is called with `uregs`, and switches on the system call number:

```
static void
syscallprint(Ureg *ureg)
{
    uint32 *sp;
    int syscallno;
    vlong offset;
    Fmt fmt;
    int len;
    uint32 argp, a;

    sp = (uint32*)(up->pmmu.uzero + ureg->usp);
    syscallno = ureg->ax;
    offset = 0;
    fmtstrinit(&fmt);
    fmtprint(&fmt, "%d %s ", up->pid, up->text);
    /* accomodate process-private system calls */

    if(syscallno > nelem(sysctab))
        fmtprint(&fmt, " %d %#x ", syscallno, sp[0]);
    else
        fmtprint(&fmt, "%s %#ux ", sysctab[syscallno], sp[0]);

    if(up->syscalltrace)
        free(up->syscalltrace);

    switch(syscallno) {
    case SYSR1:
        fmtprint(&fmt, "%08ux %08ux %08ux", sp[1], sp[2], sp[3]);
        break;
    case _ERRSTR:
        fmtuserstring(&fmt, sp[1], "");
        break;
    case BIND:
        fmtuserstring(&fmt, sp[1], " ");
        fmtuserstring(&fmt, sp[2], " ");
        fmtprint(&fmt, "%#ux", sp[3]);
        break;
    }
```

There are a few problems with this approach. Fmtuserstring does reallocation. For several reasons, realloc is not supported in the 9k kernel. The use of uregs makes this code very architecture-dependent (which is why the code is in trap.c). The 9k version is restructured and the code moved to port/:

Take it as a hint that one should not do reallocation

```
void
syscallfmt(int syscallno, va_list list)
{
    long l;
    Fmt fmt;
    void *v;
    vlong vl;
    uintptr p;
    int i[2], len;
    char *a, **argv;

    fmtstrinit(&fmt);
    fmtprint(&fmt, "%d %s ", up->pid, up->text);

    if(syscallno > nsyscall)
        fmtprint(&fmt, " %d ", syscallno);
    else
        fmtprint(&fmt, "%s ", systab[syscallno].n);

    if(up->syscalltrace != nil)
        free(up->syscalltrace);

    switch(syscallno){
    case SYSR1:
        p = va_arg(list, uintptr);
        fmtprint(&fmt, "%#p", p);
        break;
    case _ERRSTR:                /* deprecated */
    case CHDIR:
    case EXITS:
    case REMOVE:
        a = va_arg(list, char*);
        fmtuserstring(&fmt, a, "");
        break;
    case BIND:
        a = va_arg(list, char*);
        fmtuserstring(&fmt, a, " ");
        a = va_arg(list, char*);
        fmtuserstring(&fmt, a, " ");
        i[0] = va_arg(list, int);
        fmtprint(&fmt, "%#ux", i[0]);
        break;
```

In the 9k version, the uregs struct is not visible. Rather, the function is set up as a syscall number and va_args. Most of the 9k system call tracing support is in the port directory, with limited impact on the architecture directories. For example, the architecture-dependent code for supporting the print looks like this:

```
syscallfmt(scallnr, (va_list)(sp+BY2SE));
```

Note the conversion of a very machine-dependent item — the stack pointer — to a `va_list`. This conversion is very simple on some architectures, and might be more complex on others.

The only other issue with tracing is to ensure that children of `fork` inherit the tracing flag, and that the `syscalltrace` buffer and `procctl` flag are cleared on exit. These changes are in `newproc()` and `pexit()` respectively and finding them is left as an exercise.

Tracing code: `ratrace`

For reasons of space we will discuss the code that traces by a given PID. The command-based code is little different, in that it forks a child and traces that child by PID.

The code sets up three channels, one for forks, one for exits, and one for process output. It keeps track of how many processes it is reading from and, when that count goes to zero, it exits. The basic setup code is then:

```
out = chancreate(sizeof(struct Str*), 0);
quit = chancreate(sizeof(char*), 0);
forkc = chancreate(sizeof(ulong *), 0);
nread++;
procrfork(writer, nil, 8192, 0);
reader((void*)pid);
```

The 'out' channel is for process output. It takes pointers to a `Str` struct, defined as:

```
typedef struct Str Str;
struct Str {
    char *buf;
    int len;
};
```

These structs are allocated by the producer (the reader thread) for process output and freed by consumer (the writer thread).

The writer thread reads the three channels and does bookkeeping and output:

```
/* Catch exits */
a[0].op = CHANRCV;
a[0].c = quit;
a[0].v = nil;
/* Catch IO -- channel of pointers to a struct */
a[1].op = CHANRCV;
a[1].c = out;
a[1].v = &s;
/* Catch forks -- channel of ints so we know the PID */
a[2].op = CHANRCV;
a[2].c = forkc;
a[2].v = &newpid;
/* That is all */
a[3].op = CHANEND;

for(;;) {
switch(alt(a)){
/* proc exited */
case 0:
    nread--;
    if(nread <= 0)
        goto done;
    break;
case 1:
    /* output */
    fprintf(2, "%s", s->buf);
    free(s);
    break;
case 2:
    /* proc forked */
    nread++;
    break;
}
}
done:
    exits(nil);
}
```

The reader thread processes the output from the child processes, looking for rfork instantiations that create a new process.

```
void
reader(void *v)
{
    char *ctl, *truss;
    int pid, newpid;
    int cfd, tfd;
    Str *s;
    int forking = 0;

    pid = (int)v;
    ctl = smprint("/proc/%d/ctl", pid);
    if ((cfd = open(ctl, OWRITE)) < 0)
        die(smprint("%s: %r", ctl));
    truss = smprint("/proc/%d/syscall", pid);
    if ((tfd = open(truss, OREAD)) < 0)
        die(smprint("%s: %r", truss));

    cwrite(cfd, ctl, "stop", 4);
    cwrite(cfd, truss, "startsyscall", 12);
}
```

This setup gets the process into a traced state. The stop is only strictly necessary on the very first reader that is invoked, but does no harm. The process is started and then resumed, and will stop on the next system call.

The code sets up to read a buffer up to 8191 bytes (to ensure null termination) of an 8192 byte buffer. The buffer is a Str, which is what is sent down the chan.

```
s = mallocz(sizeof(Str) + 8192, 1); /* The '+' is not a typo */
s->buf = (char *)&s[1];
```

This snippet allocates contiguous memory, with the Str struct and its data. Then the code simply hangs on reading the syscalltrace fd.

```
/*
 * 8191 is not a typo. It ensures a null-terminated string.
 * The device currently limits to 4096 anyway
 */
while((s->len = pread(tfd, s->buf, 8191, 0ULL)) > 0){
    if (forking && (s->buf[1] == '=') && (s->buf[3] != '-')) {
        forking = 0;
        newpid = strtol(&s->buf[3], 0, 0);
        sendp(forkc, (void*)newpid);
        procrfork(reader, (void*)newpid, 8192, 0);
    }
}
```

This code examines the system call return value, indicated by an '=' as the first character. The 'forking' variable is set to 1 by the previous pass through this loop if the system call would create a new process. Hence: if the last system call should have created a new process; and this is a system call return, i.e. first byte is '='; and it succeeded, indicated by the third byte not being a '-' sign, i.e. the return value of the fork was not -1; then we need a new reader proc. The code then reads the pid and starts a new reader, while

also sending the new pid to the writer thread.

Catching forks is a bit tricky, this three part test has worked well:

```
/* There are three tests here to guarantee no false positives */
if (strstr(s->buf, " Rfork") != nil) {
    char *a[8];
    char *rf;
    rf = strdup(s->buf);
    if (tokenize(rf, a, 8) == 5) {
        unsigned long flags;
        flags = strtoul(a[4], 0, 16);
        if (flags & RFPROC)
            forking = 1;
    }
    free(rf);
}
sendp(out, s);
cwrite(cfd, truss, "startsyscall", 12);
s = mallocz(sizeof(Str) + 8192, 1);
s->buf = (char *)&s[1];
}
```

Once the process quits, reader sends an indication on the quit chan and exits.

```
sendp(quit, nil);
threadexitsall(nil);
}
```

Limitations

There are a few remaining problems. We made a decision early on for non-printing characters to put a '.' in the output in place of the character. That was a mistake. Information is lost when the output is displayed. We need a better data format that shows more than '.'. The '\xxx' format is not an obvious improvement; possibly symbols would work.

Currently, we only show 32 bytes. The amount of data to be shown should be controllable via a write to the ctl file of a process. Interestingly, this limitation has not yet been a problem.

Tracing slows things down quite a bit. Is this a real problem? It is hard to make a strong case either way.

Conclusions

Ratrace is a system call tracing system for Plan 9. There are two implementations, one for vx32 and one for the 9k variant of the Plan 9 kernel. We have used ratrace to debug programs and find problems with kernels. Because it presents the data to the user program as text, the nature of the information passed from kernel to user is very different from existing tools for Unix, and as a result trace programs far smaller than its Unix counterparts.

Acknowledgements

Russ Cox provided feedback and improved the vx32 version substantially. Jim McKie greatly improved the implementation of ratrace now found in the 9k kernel. Noah Evans wrote the first version of the ratrace tool.